

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

(NUCES — FAST)

MedFlow AI

An Agentic AI Assistant for Clinical Decision Support

TECHNICAL DOCUMENTATION

Version 1.0

PROJECT TEAM

Talha Habib

Abdul Ghafoor

National University of Computer and Emerging Sciences (NUCES)

Academic Project Documentation • 2026

Table of Contents

- 1. Introduction 3
 - 1.1 Overview 3
 - 1.2 Problem Statement 3
 - 1.3 Objectives 3
 - 1.4 Scope 3
 - 1.5 Out of Scope 3
- 2. System Overview 4
 - 2.1 Illustrative Scenario 4
 - 2.2 What the AI Needs to Understand 4
 - 2.3 Design Principle: Doctor-in-the-Loop 4
- 3. Technology Stack 4
 - 3.1 Stack Summary 4
 - 3.2 Technology Justifications 5
 - 3.3 Open Decisions 6
- 4. System Architecture 6
 - 4.1 High-Level Architecture 6
 - 4.2 Request Lifecycle 7
- 5. Agentic AI Design (LangGraph) 7
 - 5.1 Multi-Agent Structure 7
 - 5.2 The ReAct Reasoning Loop 8
 - 5.3 Why LangGraph Specifically 8
- 6. Knowledge Retrieval: RAG and GraphRAG 8
 - 6.1 RAG — Retrieval-Augmented Generation 8
 - 6.2 GraphRAG — Graph-Based Retrieval 9
 - 6.3 RAG vs. GraphRAG — When Each Is Used 9
- 7. Tool Integration Layer (MCP) 9
 - 7.1 Exposed MCP Tools 9
 - 7.2 Benefits of the MCP Approach 10
- 8. Workflow Automation Layer (n8n) 10
 - 8.1 Example Workflow: Follow-Up Appointment 10
 - 8.2 Why Automation Is Separated From the Agent Layer 10
- 9. Data Design 10
 - 9.1 MySQL — Relational Schema 10
 - 9.2 Neo4j — Node & Relationship Types 11
 - 9.3 Pinecone — Vector Index 11
 - 9.4 Data Privacy Note 11
- 10. API Design (FastAPI, REST) 11
 - 10.1 API Design Principles 12
- 11. Frontend — Doctor Dashboard 12
 - 11.1 Dashboard Sections 12
 - 11.2 Example Interactions 12
- 12. Security, Safety & Compliance 12
 - 12.1 Doctor-in-the-Loop Safeguard 12
 - 12.2 Authentication & Access Control 13
 - 12.3 Auditability 13
 - 12.4 Data Handling 13
 - 12.5 Known Limitations for a Production Setting 13
- 13. Deployment Architecture 13
 - 13.1 Containerized Services 13
 - 13.2 Deployment Flow 13
- 14. Future Work 14
- 15. Conclusion 14

1. Introduction

1.1 Overview

MedFlow AI is an agentic, AI-powered clinical assistant designed to help doctors quickly understand a patient's complete medical history, retrieve relevant medical evidence, and safely automate approved hospital workflows. Rather than replacing clinical judgment, MedFlow AI acts as an intelligent research and coordination layer that sits between fragmented hospital data and the doctor, surfacing the right information at the right time and executing routine tasks only after explicit doctor approval.

1.2 Problem Statement

Clinicians routinely work with patients who have years of accumulated medical data spread across multiple systems: previous diagnoses, lab results, prescriptions, allergy records, and visit histories. Manually reviewing this information is time-consuming, error-prone, and takes valuable time away from direct patient care. At the same time, doctors are expected to stay current with medical guidelines and research, which is difficult without dedicated support tools. There is a clear need for a system that can consolidate patient data, connect it with relevant medical knowledge, and support (not replace) clinical decision-making.

1.3 Objectives

- Consolidate a patient's fragmented medical history into a single, easy-to-read summary generated on demand.
- Surface relevant medical guidelines and research evidence alongside patient-specific findings.
- Model relationships between a patient's conditions, medications, allergies, and visits using a graph-based approach (GraphRAG).
- Provide a multi-agent reasoning pipeline (LangGraph) that separates patient-data analysis, medical evidence retrieval, and medication/allergy safety checks.
- Allow the AI to safely interact with hospital systems (patient database, calendar, email) through a standardized tool-calling protocol (MCP).
- Automate approved, low-risk administrative workflows such as appointment scheduling and follow-up reminders (n8n).
- Keep the doctor firmly in the loop — the AI investigates and recommends; the doctor decides and approves.

1.4 Scope

The scope of this project covers the design and implementation of a working prototype consisting of: a React/Next.js doctor-facing dashboard, a FastAPI backend, a multi-agent orchestration layer built with LangGraph, a Retrieval-Augmented Generation (RAG) pipeline over medical literature, a GraphRAG layer over patient relationship data, an MCP tool layer for hospital-system integration, and an n8n-based automation layer for approved workflows. The system is intended as an academic demonstration and is not certified for real clinical deployment.

1.5 Out of Scope

- Autonomous diagnosis or treatment decisions without doctor review.
- Integration with a real, production hospital Electronic Health Record (EHR) system — a representative/sample dataset is used instead.
- Regulatory certification (e.g., HIPAA, FDA/DRAP) required for real-world clinical use.

2. System Overview

MedFlow AI can be understood as an AI assistant sitting beside the doctor. Instead of manually scrolling through years of patient records, the doctor asks a natural-language question and the system investigates, cites its evidence, and flags anything requiring attention.

2.1 Illustrative Scenario

Example Query Scenario
<i>Consider a patient, Ali, who has a documented history of diabetes and hypertension, is on multiple medications, has a known drug allergy, and has visited the hospital five times previously. When the doctor asks MedFlow AI to “summarize Ali’s medical history and highlight anything I should pay attention to,” the system retrieves the patient’s structured history, cross-references active medications against known allergies, pulls supporting medical guidelines, and returns a structured summary with an explicit safety warning — leaving the final clinical decision to the doctor.</i>

2.2 What the AI Needs to Understand

- Patient-specific data — the relationships between a patient's conditions, medications, allergies, lab results, and visit history. Handled by GraphRAG over a Neo4j graph.
- Medical knowledge — guidelines, research papers, and drug references. Handled by a RAG pipeline over a Pinecone vector index.
- Hospital tools — the patient database, appointment system, calendar, and email, accessed through MCP (Model Context Protocol).

2.3 Design Principle: Doctor-in-the-Loop

Every output the system produces that could influence patient care is explicitly labeled as requiring doctor review. Automated actions (such as booking a follow-up appointment) are only triggered after the doctor has explicitly approved the underlying recommendation — the AI never acts unilaterally on clinical matters.

3. Technology Stack

The technology choices below were selected for this project based on team discussion, balancing rapid development, self-hosting flexibility (important for data-privacy constraints), and strong support for agentic AI and graph-based retrieval workloads.

3.1 Stack Summary

Layer	Technology	Purpose
Frontend	Next.js (React)	Doctor-facing dashboard with server-side rendering and file-based routing
Backend API	FastAPI (Python)	REST API layer connecting the frontend to the agentic AI pipeline
Agent Orchestration	LangGraph	Coordinates the multi-agent (Patient / Medical / Safety) reasoning pipeline
Reasoning Pattern	ReAct (Reason + Act)	Think → choose tool → use tool → observe result → continue loop
LLM Inference	Groq	Low-latency inference for agent reasoning and summarization steps
Relational Database	MySQL	Stores structured hospital data: patients, appointments, users, audit logs
Graph Database	Neo4j	Stores patient-entity relationships for GraphRAG
Vector Database	Pinecone	Stores embeddings of medical guidelines and research literature for RAG
Tool Integration	MCP (Model Context Protocol)	Standardized interface exposing hospital systems as callable AI tools
Workflow Automation	n8n	Executes approved multi-step workflows (appointment booking, reminders, emails)
Containerization	Docker / Docker Compose	Packages each service for consistent deployment
Deployment Target	On-premise / University server	Hosts the Dockerized stack within a controlled environment
API Style	REST	Communication contract between the Next.js frontend and the FastAPI backend

3.2 Technology Justifications

3.2.1 Next.js

Chosen over a plain Vite/React SPA because it provides built-in routing, server-side rendering for faster initial dashboard loads, and a more structured project layout — useful for a multi-page doctor dashboard.

3.2.2 FastAPI

Provides asynchronous request handling, automatic OpenAPI documentation, and native Python support, which is important since LangGraph, the RAG pipeline, and the MCP client are all Python-based.

3.2.3 MySQL

Used as the system of record for structured, transactional hospital data — patient demographics, appointments, user accounts, and audit trails — where strict relational integrity is required.

3.2.4 Neo4j (GraphRAG)

Patient medical histories are naturally relational. Neo4j's native graph queries (Cypher) make it efficient to traverse multi-hop relationships — e.g., “find all medications currently prescribed to this patient that conflict with a documented allergy”.

3.2.5 Pinecone (RAG)

Medical guidelines and research papers are unstructured text. Pinecone stores vector embeddings of this literature for semantic similarity search, retrieving the most relevant guideline passages for a given clinical question.

3.2.6 Groq

Groq's inference infrastructure was chosen for its low-latency response times, which matters in a multi-agent pipeline where several LLM calls may be chained sequentially for a single doctor query.

3.2.7 LangGraph

Provides a graph-based framework for defining and running the multi-agent workflow as an explicit state machine. Each agent's responsibility is explicit, testable, and easier to reason about.

3.2.8 MCP

Standardizes how the AI agents call external tools (patient database lookups, calendar checks, email sending) instead of writing bespoke integration code for every agent-tool pair.

3.2.9 n8n

Chosen as the automation engine for approved, doctor-triggered workflows because it provides a visual, auditable workflow builder that can orchestrate multi-step processes without embedding this logic directly in application code.

3.2.10 Docker + On-Premise Hosting

Given the sensitivity of medical data, even sample/synthetic data, the team chose to containerize the entire stack with Docker and deploy it to a university/on-premise server to keep the deployment self-contained and reproducible.

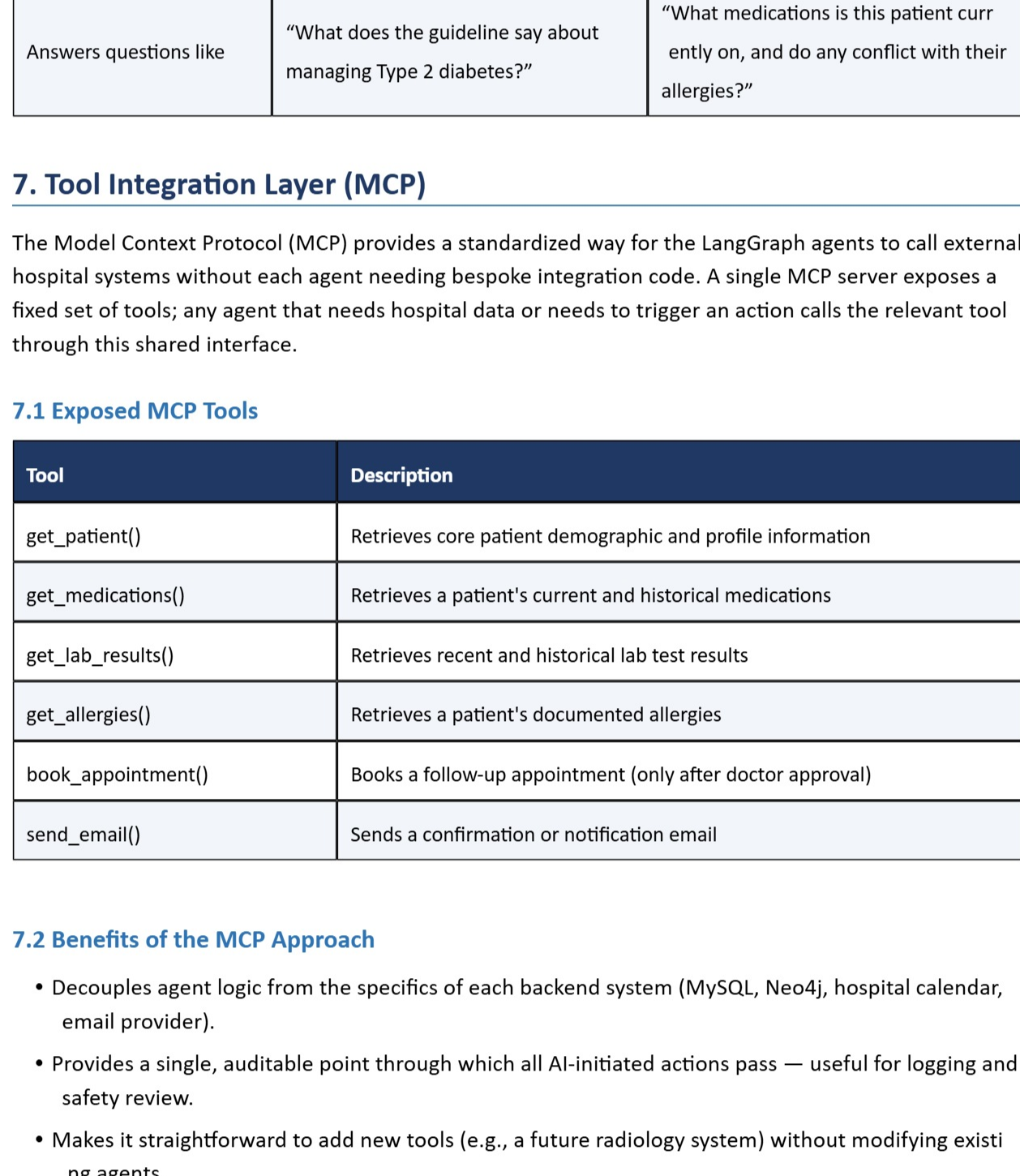
3.3 Open Decisions

Authentication strategy for doctor login has not been finalized. JWT-based session authentication is the current front-runner given the REST API design and stateless FastAPI backend; OAuth2 integration with hospital identity providers remains a possible extension. This decision, along with role-based access control (RBAC) design, is documented as an open item to be resolved during the design phase.

4. System Architecture

The architecture follows a layered design: a presentation layer (Next.js), an API layer (FastAPI), an agentic reasoning layer (LangGraph), a knowledge/retrieval layer (RAG + GraphRAG), a tool-access layer (MCP), and an automation layer (n8n).

4.1 High-Level Architecture



4.2 Request Lifecycle

- Doctor logs into the Next.js dashboard and opens the AI Assistant panel for a specific patient.
- Doctor asks: “Summarize Ali’s medical history and tell me what I should pay attention to.”
- The Next.js frontend sends a REST request to the FastAPI backend.
- FastAPI passes the query into the LangGraph orchestrator, which routes it to the Patient, Medical, and Safety agents.
- The Patient Agent queries Neo4j (via an MCP tool call) to retrieve Ali’s conditions, medications, allergies, and visit history as a connected graph.
- The Medical Agent queries Pinecone for relevant guideline passages related to Ali’s conditions.
- The Safety Agent cross-references Ali’s active medications against his documented allergies and flags a conflict.
- The orchestrator combines all three agents’ outputs into a single structured summary, explicitly marked “Doctor Review Required.”
- FastAPI returns the structured response to the frontend, which renders it in the AI Summary panel.
- If the doctor approves a follow-up action (e.g., scheduling a visit), the backend triggers an n8n workflow that books the appointment and sends a confirmation.

5. Agentic AI Design (LangGraph)

5.1 Multi-Agent Structure

Rather than using a single large prompt to handle every type of request, MedFlow AI splits responsibility across specialized agents coordinated by LangGraph. This mirrors how a real clinical team might divide labor and makes the system’s reasoning transparent and auditable.

Agent	Responsibility	Primary Data Source
Main / Orchestrator Agent	Interprets the doctor’s request and decides which sub-agents to invoke and in what order	N/A — routing logic
Patient Agent	Retrieves and structures the patient’s own history and relationships	Neo4j (GraphRAG)
Medical Agent	Retrieves relevant guidelines, research, and drug information	Pinecone (RAG)
Safety Agent	Cross-checks current medications, dosages, and allergies for conflicts	MySQL + Neo4j

5.2 The ReAct Reasoning Loop

Each agent follows the ReAct (Reason + Act) pattern when handling a task: it reasons about what information it needs, selects an appropriate tool, invokes that tool, observes the result, and either continues reasoning or produces a final answer.

Stage	Description
Think	“I need Ali’s latest lab results.”
Choose Tool	get_lab_results()
Act	Call the tool via MCP
Observe	Receive structured lab data
Respond	Incorporate result into the final answer

5.3 Why LangGraph Specifically

LangGraph models the multi-agent workflow as an explicit graph of states and transitions rather than an implicit chain. This allows the orchestrator to conditionally route between agents (e.g., skip the Safety Agent if no medications are involved), retry a step on tool failure, and maintain shared state across the whole pipeline — all of which are difficult to express reliably in a single linear prompt chain.

6. Knowledge Retrieval: RAG and GraphRAG

6.1 RAG — Retrieval-Augmented Generation

RAG is used to ground the Medical Agent’s responses in real, citable medical literature rather than relying purely on the LLM’s internal, potentially outdated knowledge. Guideline documents and research papers are chunked, embedded, and stored in Pinecone. When the Medical Agent needs supporting evidence, it embeds the doctor’s query and performs a similarity search against Pinecone to retrieve the most relevant passages, which are then included in the LLM’s context before it generates a response.

In short: RAG = search medical knowledge before answering.

6.2 GraphRAG — Graph-Based Retrieval

Where RAG is well suited to unstructured text, patient data is inherently relational — a single patient connects to multiple conditions, medications, lab results, and visits, and those entities connect to each other. GraphRAG stores this as a property graph in Neo4j, allowing the Patient Agent to traverse relationships directly via Cypher queries rather than reconstructing them from multiple relational joins or unstructured notes.

6.2.1 Example Graph Structure

(A11) —[:HAS_CONDITION]→ (Diabetes)

(A11) —[:HAS_CONDITION]→ (Hypertension)

(A11) —[:PRESCRIBED]→ (Medicine A)

(A11) —[:PRESCRIBED]→ (Medicine B)

(A11) —[:HAS_ALLERGY]→ (Drug X)

(A11) —[:HAD_VISIT]→ (Visit 1..5)

(Medicine B) —[:CONTRAINDICATED_WITH]→ (Drug X)

6.3 RAG vs. GraphRAG — When Each Is Used

Aspect	RAG (Pinecone)	GraphRAG (Neo4j)
Data type	Unstructured text (guidelines, papers)	Structured entities and relationships (patient data)
Query style	Semantic similarity search	Graph traversal (Cypher pattern matching)
Used by	Medical Agent	Patient Agent, Safety Agent
Answers questions like	“What does the guideline say about managing Type 2 diabetes?”	“What medications is this patient currently on, and do any conflict with their allergies?”

7. Tool Integration Layer (MCP)

The Model Context Protocol (MCP) provides a standardized way for the LangGraph agents to call external hospital systems without each agent needing bespoke integration code. A single MCP server exposes a fixed set of tools; any agent that needs hospital data or needs to trigger an action calls the relevant tool through this shared interface.

7.1 Exposed MCP Tools

Tool	Description
get_patient()	Retrieves core patient demographic and profile information
get_medications()	Retrieves a patient’s current and historical medications
get_lab_results()	Retrieves recent and historical lab test results
get_allergies()	Retrieves a patient’s documented allergies
book_appointment()	Books a follow-up appointment (only after doctor approval)
send_email()	Sends a confirmation or notification email

7.2 Benefits of the MCP Approach

- Decouples agent logic from the specifics of each backend system (MySQL, Neo4j, hospital calendar, email provider).
- Provides a single, auditable point through which all AI-initiated actions pass — useful for logging and safety review.
- Makes it straightforward to add new tools (e.g., a future radiology system) without modifying existing agents.

8. Workflow Automation Layer (n8n)

n8n handles multi-step administrative workflows that are triggered only after a doctor has explicitly approved an AI recommendation. This keeps automation strictly bounded to non-clinical, low-risk actions.

8.1 Example Workflow: Follow-Up Appointment

- Doctor approves: “Schedule Ali’s follow-up appointment next week.”
- The FastAPI backend triggers the corresponding n8n workflow via webhook.
- n8n checks the doctor’s calendar availability.
- n8n checks the patient’s stated availability preferences.
- n8n books the appointment slot in the scheduling system.
- n8n sends a confirmation email to the patient.
- n8n schedules an automated reminder ahead of the appointment.

8.2 Why Automation Is Separated From the Agent Layer

Keeping multi-step automation logic in n8n (rather than embedding it directly inside a LangGraph agent) makes each workflow visually inspectable and independently testable, and ensures that only explicitly approved, well-defined workflows can execute. The AI agents themselves never have direct write access to scheduling or communication systems; they can only request an n8n workflow after doctor approval.

9. Data Design

9.1 MySQL — Relational Schema

Table	Purpose	Key Fields
patients	Core patient records	patient_id, name, dob, contact_info
users	Doctor / staff accounts	user_id, name, role, credentials
appointments	Scheduled visits	appointment_id, patient_id, doctor_id, datetime, status
audit_log	Records every AI-initiated action for traceability	log_id, agent_name, action, timestamp, approved_by

9.2 Neo4j — Node & Relationship Types

Nodes: Patient, Condition, Medication, Allergy, Visit, LabResult

Relationships: HAS_CONDITION, PRESCRIBED, HAS_ALLERGY, HAD_VISIT, CONTRAINDICATED_WITH, PRODUCED_RESULT

9.3 Pinecone — Vector Index

- Index contents: chunked medical guidelines, research abstracts, and drug reference sheets.
- Each vector is stored with metadata (source, publication, section) to support citation in the Medical Agent’s responses.

9.4 Data Privacy Note

Data Privacy
<i>Because real patient data is sensitive, the prototype uses synthetic or de-identified sample datasets. Even so, the system is designed with the same data-handling discipline expected of a production clinical tool: access-controlled databases, an audit log of all AI-initiated actions, and on-premise (rather than public cloud) hosting.</i>

10. API Design (FastAPI, REST)

The frontend and backend communicate over a REST API exposed by FastAPI. Representative endpoints are summarized below; exact routes may evolve during implementation.

Endpoint	Method	Description
/api/auth/login	POST	Authenticates a doctor/staff user
/api/patients/{id}	GET	Retrieves a patient’s core profile
/api/patients/{id}/summary	POST	Triggers the multi-agent pipeline to generate an AI summary
/api/appointments	GET / POST	Lists or creates appointments
/api/appointments/{id}/approve	POST	Doctor approval that triggers the n8n booking workflow
/api/knowledge/search	GET	Direct RAG search over the medical knowledge base

10.1 API Design Principles

- Statelessness: each request carries the authentication token required to process it, in line with the REST style and a JWT-based auth model.
- Every AI-generated clinical output is returned with an explicit review flag rather than being presented as a final answer.
- Endpoints that trigger real-world actions (appointment booking, email) require an explicit doctor-approved step before execution.

11. Frontend — Doctor Dashboard

The dashboard is built with Next.js and organized around the doctor’s typical workflow: reviewing a patient, consulting the AI assistant, and acting on its recommendations.

11.1 Dashboard Sections

Section	Purpose
Dashboard	At-a-glance overview of the doctor’s day, pending approvals, and alerts
Patients	Searchable patient list and individual patient profiles
AI Assistant	Conversational interface for asking natural-language questions about a patient
Appointments	Calendar view of scheduled and pending appointments
Reports	Generated patient summaries and exportable reports
Knowledge	Direct search interface over the medical knowledge base (RAG)
Workflows	Visibility into automated (n8n) workflows and their status

11.2 Example Interactions

- “Summarize this patient’s history.”
- “Show abnormal lab results.”
- “Find relevant medical guidelines.”
- “Schedule follow-up.”
- “Generate a patient report.”

12. Security, Safety & Compliance

12.1 Doctor-in-the-Loop Safeguard

No clinical output from the AI pipeline is treated as final. Every summary, recommendation, or flagged risk is explicitly labeled “Doctor Review Required,” and no clinically consequential action is taken without the doctor’s affirmative approval.

12.2 Authentication & Access Control

Authentication for doctor login was still an open decision at the time of writing. JWT-based authentication is the current leading approach given the REST/FastAPI design; role-based access control (RBAC) is planned so that only authorized clinical staff can view patient data or approve AI-triggered actions.

12.3 Auditability

Every tool call made by an agent through MCP, and every action executed by n8n, is recorded in the MySQL audit_log table, capturing which agent acted, what action was taken, when, and under whose approval — supporting traceability for both academic evaluation and, in a production context, regulatory review.

12.4 Data Handling

- Use of synthetic/de-identified data for the prototype.
- On-premise Docker deployment rather than public cloud, reducing external data exposure during development.
- Clear separation between read-only AI tools (get_patient, get_lab_results, etc.) and write/action tools (book_appointment, send_email) with the latter gated behind doctor approval.

12.5 Known Limitations for a Production Setting

This prototype is an academic demonstration. A real clinical deployment would additionally require: formal regulatory compliance review (e.g., HIPAA or the applicable local health-data regulation), integration with certified hospital EHR systems, penetration testing, encryption-at-rest and in-transit for all data stores, and a formal clinical validation process for any AI-generated recommendation.

13. Deployment Architecture

All services are containerized with Docker and orchestrated via Docker Compose, then deployed to an on-premise or university-hosted server rather than a public cloud provider.

13.1 Containerized Services

Service	Container
Frontend	Next.js app (Node runtime)
Backend API	FastAPI app (Python/Uvicorn)
Relational DB	MySQL container with persistent volume
Graph DB	Neo4j container with persistent volume
Automation Engine	n8n container

Note: Pinecone and Groq are used as managed, external services accessed via API from the FastAPI backend, rather than being self-hosted containers, since both are cloud-native offerings.

13.2 Deployment Flow

- Each service is built into its own Docker image using a dedicated Dockerfile.
- docker-compose.yml defines the multi-container application, including networking between the frontend, backend, MySQL, Neo4j, and n8n.
- Environment variables (API keys for Groq and Pinecone, database credentials) are injected via a secured .env file, excluded from version control.
- The full stack is deployed on the on-premise/university server, with persistent volumes for MySQL and Neo4j to preserve data across restarts.

14. Future Work

- Finalize and implement the authentication strategy (JWT, with optional hospital SSO/OAuth2 integration).
- Integrate with a real (or more realistic sandboxed) hospital EHR system for broader validation.
- Expand the Safety Agent to cover drug-drug interactions in addition to drug-allergy conflicts.
- Add a feedback loop where doctors can rate AI summaries to improve future retrieval and ranking.
- Extend the Knowledge module to support uploading and indexing institution-specific clinical protocols.
- Formal usability testing with clinical staff to refine the dashboard workflow.

15. Conclusion

MedFlow AI demonstrates how modern agentic AI techniques — multi-agent orchestration with LangGraph, the ReAct reasoning pattern, Retrieval-Augmented Generation, graph-based retrieval (GraphRAG), standardized tool access via MCP, and workflow automation via n8n — can be combined into a single, coherent product aimed at a real-world problem: helping doctors navigate complex patient histories without letting the AI make critical decisions on its own.

The project intentionally keeps the doctor at the center of every decision, using AI to investigate, connect, and prepare information rather than to diagnose or prescribe. This makes MedFlow AI both a technically rich demonstration of agentic AI, RAG, GraphRAG, MCP, and automation, and a responsibly scaled academic project.

Project Team

Role	Name
Team Member	Talha Habib
Team Member	Abdul Ghafoor